

Rapport de benchmark

BenchmarkTick-16 — analyse CPU, mémoire et hyperfine

Généré le	2026-09-24T08:48:09.5442216Z
Commande	go test ./game -run '^\$' -bench '^BenchmarkTick\$' -benchmem -count 1
Plateforme	windows/amd64
Processeur	AMD Ryzen 7 7735HS with Radeon Graphics
Profil CPU	cpu-20260924-084809666.prof
Profil mémoire	memory-20260924-084809666.prof

1. Résultats du benchmark

Benchmark	Moyenne	ns/op	B/op	allocs/op
BenchmarkTick-16	6.74 ms	6738229	59034	17

Détail hyperfine

Métrique	Valeur
Moyenne	6.0328 s
Écart-type	n/a
Médiane	6.0328 s
Temps utilisateur	28.9481 s
Temps système	1.5872 s
Min	6.0328 s
Max	6.0328 s

Résumé CPU et mémoire

Mesure	Valeur
CPU échantillonné (pprof)	27.38s
Mémoire allouée (pprof)	102.38MB
Mémoire par opération	59034 B/op

Garbage collector Go

Cycles GC	68
Temps GC cumulé	72.300 ms

Plus longue pause	4.000 ms
Temps CPU GC cumulé	352.440 ms
Objectif mémoire maximal	95.00 MB

2. Profil CPU — où passe le temps ?

Type de profil : **cpu**. Total échantillonné : **27.38s**.

Fonction	Flat	Flat %	Cumulé	Cumulé %
jeu-de-la-vie/game.(*Board).infectionCandidate	22.45s	81.99%	25.60s	93.50%
jeu-de-la-vie/game.(*Board).index (inline)	2.54s	9.28%	2.59s	9.46%
jeu-de-la-vie/game.(*Board).Step	0.98s	3.58%	1.44s	5.26%
runtime.asyncPreempt	0.38s	1.39%	0.38s	1.39%
math/rand.(*Rand).Float64 (inline)	0.26s	0.95%	0.44s	1.61%
jeu-de-la-vie/game.squaredDistance (inline)	0.23s	0.84%	0.24s	0.88%
jeu-de-la-vie/game.(*Board).countInfectionCand	0.17s	0.62%	25.78s	94.16%
math/rand.(*Rand).Int63 (inline)	0.03s	0.11%	0.18s	0.66%

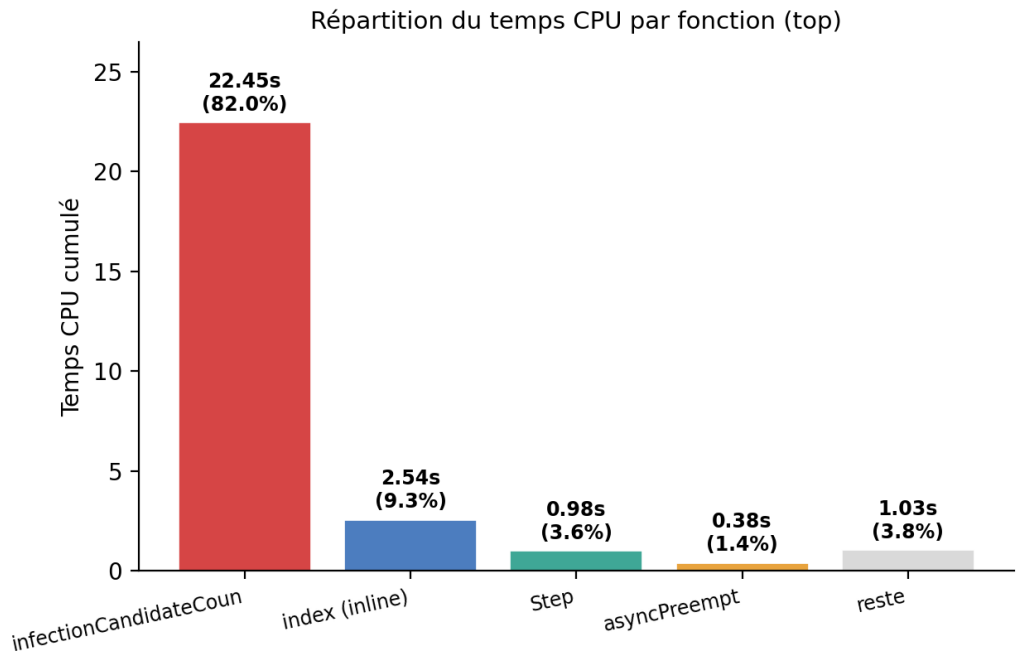


Figure — Répartition du temps CPU par fonction (temps flat, top fonctions).

[illegible]

2.1 Diagnostic automatique

- Le nom de cette fonction suggère une recherche/un parcours répété (distance, voisinage, scan...) — vérifier si elle itère sur l'ensemble des données à chaque appel plutôt que sur un sous-ensemble borné (structure spatiale : grille, quadtree, index).
- Comparer ce hotspot avec le run précédent : s'il a changé de fonction, l'optimisation précédente a fonctionné et un nouveau goulot est apparu ailleurs.
- Vérifier s'il existe un algorithme en $O(n)$ ou $O(\log n)$ équivalent pour cette opération avant d'optimiser au niveau micro (inlining, allocations).

Rappel — pourquoi la complexité algorithmique domine
à mesure que les données grandissent (schéma, non mesuré)

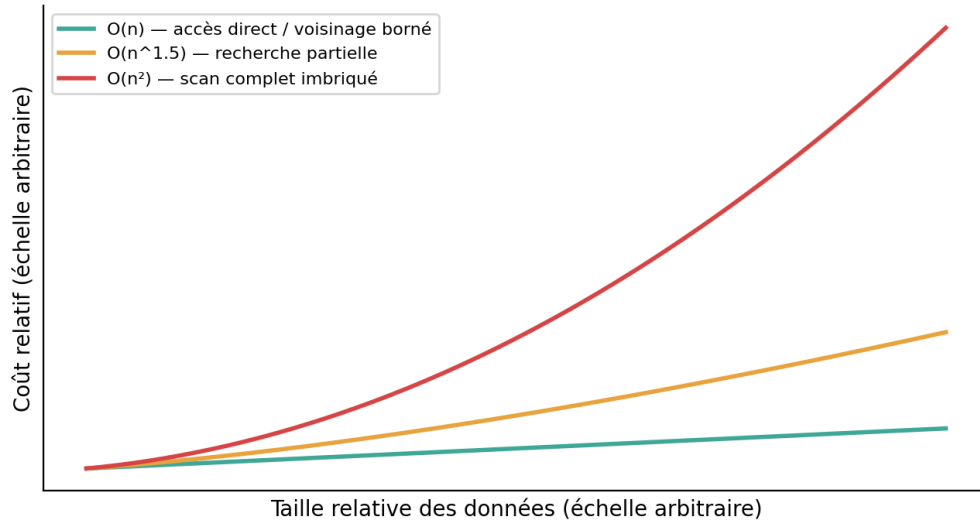


Figure — Rappel pédagogique sur la complexité algorithmique (schéma générique).

3. Profil mémoire — allocations

Type de profil : `alloc_space`. Total : **102.38MB**.

Fonction	Flat	Flat %	Cumulé	Cumulé %
jeu-de-la-vie/game.(*Board).ensureScratch (inl	52.47MB	51.25%	52.47MB	51.25%
jeu-de-la-vie/game.NewBoard (inline)	20.72MB	20.23%	20.72MB	20.23%
jeu-de-la-vie/game.RandomBoard	16.57MB	16.18%	37.28MB	36.42%
runtime.mallocgc	5.01MB	4.89%	5.01MB	4.89%
jeu-de-la-vie/game.(*Board).countInfectionCand	2MB	1.95%	2MB	1.95%
runtime/pprof.StartCPUProfile	1.16MB	1.13%	1.16MB	1.13%
math/rand.newSource (inline)	1.01MB	0.98%	1.01MB	0.98%
compress/flate.NewWriter (inline)	0.88MB	0.86%	1.45MB	1.42%

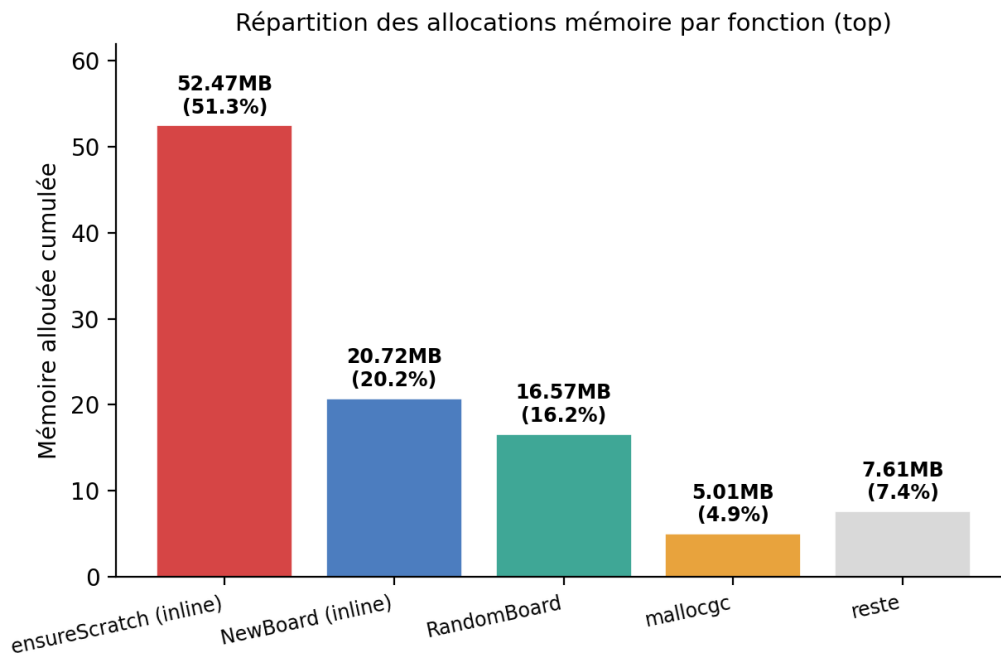


Figure — Répartition des allocations mémoire par fonction.

```
File: game.test.exe
Build ID: C:\Users\saiou\JeuDeLaVie\game.test.exe2026-09-24 10:47:52.1307629 +0200 CEST
Type: alloc_space
Time: 2026-09-24 10:48:14 CEST
Showing nodes accounting for 94.76MB, 92.56% of 102.38MB total
Dropped 20 nodes (cum <= 0.51MB)
Showing top 10 nodes out of 39

See https://git.io/JFYMW for how to read the graph
```



Figure — Graphe d'appels mémoire (go tool pprof -png).

4. Limites de ce benchmark & informations manquantes

Générées automatiquement à partir de la configuration détectée :

- **Peu de runs (1).** Avec moins de 5 exécutions, aucune médiane/écart-type fiable n'est disponible. Recommandé : ≥ 10 runs (-count=10 ou hyperfine --min-runs 10).
- **Aucun warmup.** La première exécution peut être biaisée (cache froid, JIT du GC non stabilisé). Ajouter 1-2 warmups avant la mesure.
- **Pas de comparaison avant/après** intégrée à ce rapport — conservez les PDF successifs pour comparer les hotspots et le temps moyen dans le temps.
- **Un seul point de taille de données.** Si applicable, mesurer plusieurs tailles permettrait de vérifier empiriquement la complexité algorithmique observée en section 2.

Notes de l'outil de benchmark

- Mesure du calcul du tick sur une carte 600×600 .
- La carte et le générateur aléatoire utilisent toujours la seed 42.
- La latence API est mesurée à part dans le frontend, sans rendu canvas.
- La trace GC détaillée est enregistrée dans le fichier `gc-\*.log` et résumée dans le PDF.
- Les métriques I/O, réseau et base de données ne sont pas instrumentées dans ce benchmark.

Rapport généré automatiquement le 2026-09-24 10:48:25 par generate_benchmark_report.py, à partir des fichiers les plus récents du dossier de benchmarks. Analyse réalisée avec go tool pprof.